

Generate_Synthetic_CustomerNotebook

New notebook

Generate the sample staging customer data to test the SCD

Generating this synthetic customer data as staging data for our production template for SCD Type 2 handling of our dimensions

Because we have two separate areas, here we generate the data to the staging, and in Slowly Changing Dim Notebooks, we can run the **MERGE**

We can change values to test here, and then go to the other notebook to test

```
In [1]: # Welcome to your new notebook
# Type here in the cell editor to add code!
from pyspark.sql import SparkSession
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

StatementMeta(, 28c60931-c233-4af9-8697-0a67b92fc1fd, 3, Finished, Available, Finished, False)

Generate Synthetic Staging Customer Data

We are generating random values of 200 rows, change num_samples to however value for rows you desire

```
In [2]: # Start Spark session
spark = SparkSession.builder.getOrCreate()

# Parameters
num_samples = 200

# Helper functions
def random_date(start, end):
    """Generate a random datetime between start and end"""
    return start + timedelta(seconds=np.random.randint(0, int((end - start).total_seconds())))

# Generate data as a pandas DataFrame first
np.random.seed(42)

customer_ids = [f"C{i:04d}" for i in range(1, num_samples+1)]
customer_names = [f"Customer_{i:04d}" for i in range(1, num_samples+1)]
customer_statuses = np.random.choice(["Active", "Inactive"], size=num_samples, p=[0.5, 0.5])
addresses = [f"{i} Main St" for i in range(1, num_samples+1)]
effective_start = [random_date(datetime(2020,1,1), datetime(2024,1,1)) for _ in range(num_samples)]
effective_end = [None]*num_samples
is_current = [True]*num_samples

pdf = pd.DataFrame({
    "customer_id": customer_ids,
    "customer_name": customer_names,
    "customer_status": customer_statuses,
```

```

    "address": addresses,
    "effective_start": effective_start,
    "effective_end": effective_end,
    "is_current": is_current
  })

```

StatementMeta(, 28c60931-c233-4af9-8697-0a67b92fc1fd, 4, Finished, Available, Finished, False)

Convert to Spark DataFrame - I want to see the schema

```

In [ ]: df_source = spark.createDataFrame(pdf)
df_source.printSchema()
df_source.show(5)

```

Writing to the staging folder as Delta File - This file would be written to your lakehouse/Files/Staging/customers path

```

In [4]: source_path = "Files/staging/customers"

df_source.write.format("delta") \
    .mode("overwrite") \
    .save(source_path)

```

StatementMeta(, 28c60931-c233-4af9-8697-0a67b92fc1fd, 6, Finished, Available, Finished, False)

Spark writes Delta files into:

Files/staging/customers/ |— _delta_log/ ## Delta transaction log |— part-00000-xxxx.parquet |— part-00001-xxxx.parquet

Each .parquet file contains a subset of the 200 rows (partitioning and file size depend on Spark's default).

The _delta_log folder tracks:

- When the files were written
- Schema
- Transaction versions (for ACID and time travel)

```

In [ ]: df_source.show(40)

```

Then run the Test SCD Staging data modification which we can now test our **Slowly Changing Dim Notebook** to test SCD Type 2

```

In [ ]: ta

```